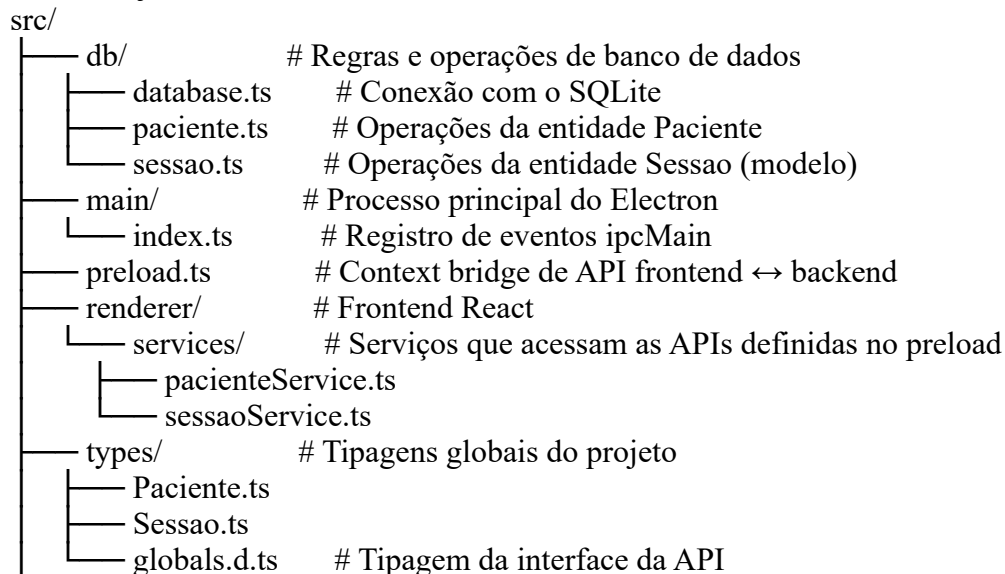


Criação de Entidades - Equilibra Manager

Objetivo

Este documento detalha todo o processo necessário para definir e implementar novas entidades no banco de dados local do projeto Equilibra Manager, que utiliza Electron + React + TypeScript no frontend e Better-SQLite3 como banco de dados local embutido. O padrão seguido é o mesmo usado na criação da entidade Sessao, e serve como referência para a adição de qualquer nova entidade, como Financeiro, Relatorio, Consulta, etc.

Estrutura do Projeto



Etapas de Criação

1. Definir Interface (src/types/)

Criar arquivo .ts com os campos

Ex: Sessao.ts, Consulta.ts Exemplo: src/types/Consulta.ts

```
export interface Consulta {
  id: number;
  paciente: number;
  data: string; // ou Date
  tipo: string;
  status: string;
  anotacoes?: string;
  paciente_nome: string; // JOIN com Paciente
}
```

2. Criar Operações DB (src/db/)

criar, listar, atualizar, deletar

Usa `getDb()` de database.ts Exemplo: [src/db/consulta.ts](#)

```
import { getDb } from '../database';
import { Consulta } from '../types/Consulta';
export function criarConsulta(consulta: Consulta): Consulta {
  const db = getDb();
  const stmt = db.prepare(
    `INSERT INTO consultas (paciente_id, data, tipo, status, anotacoes)
    VALUES (?, ?, ?, ?, ?)`
  );
  const result = stmt.run(
    consulta.paciente,
    consulta.data,
    consulta.tipo,
    consulta.status,
    consulta.anotacoes ?? null
  );
  return { ...consulta, id: Number(result.lastInsertRowid) };
```

```

}
export function listarConsultasPorPaciente(pacienteId: number): Consulta[] {
  const db = getDb();
  const stmt = db.prepare(
    `SELECT c.*, p.nome_completo AS paciente_nome
    FROM consultas c
    JOIN pacientes p ON p.id = c.paciente_id
    WHERE paciente_id = ?`
  );
  return stmt.all(pacienteId);
}
export function listarTodasConsultas(): Consulta[] {
  const db = getDb();
  const stmt = db.prepare(
    `SELECT c.*, p.nome_completo AS paciente_nome
    FROM consultas c
    JOIN pacientes p ON p.id = c.paciente_id`
  );
  return stmt.all();
}
export function atualizarConsulta(consulta: Consulta): void {
  const db = getDb();
  const stmt = db.prepare(
    UPDATE consultas
    SET data = ?, tipo = ?, status = ?, anotacoes = ?
    WHERE id = ?
  );
  stmt.run(consulta.data, consulta.tipo, consulta.status, consulta.anotacoes, consulta.id);
}
export function deletarConsulta(id: number): void {
  const db = getDb();
  const stmt = db.prepare(`DELETE FROM consultas WHERE id = ?`);
  stmt.run(id);
}

```

3. Registrar ipcMain (src/main/index.ts)

Registrar ipcMain.handle() para cada função Exemplo: src/index.ts

```

import {
  criarConsulta,
  listarConsultasPorPaciente,
  listarTodasConsultas,
  atualizarConsulta,
  deletarConsulta
} from '../db/consulta';
ipcMain.handle('criarConsulta', (_, dados) => criarConsulta(dados));
ipcMain.handle('listarConsultasPorPaciente', (_, id) => listarConsultasPorPaciente(id));
ipcMain.handle('listarTodasConsultas', () => listarTodasConsultas());
ipcMain.handle('atualizarConsulta', (_, dados) => atualizarConsulta(dados));
ipcMain.handle('deletarConsulta', (_, id) => deletarConsulta(id));

```

4. Atualizar preload.ts

Expose com contextBridge

Usa ipcRenderer.invoke Exemplo: src/preload.ts

```

contextBridge.exposeInMainWorld('api', {
  ...outrasAPIs,
  criarConsulta: (consulta) => ipcRenderer.invoke('criarConsulta', consulta),
  listarConsultasPorPaciente: (id) => ipcRenderer.invoke('listarConsultasPorPaciente', id),
  listarTodasConsultas: () => ipcRenderer.invoke('listarTodasConsultas'),
  atualizarConsulta: (consulta) => ipcRenderer.invoke('atualizarConsulta', consulta),
  deletarConsulta: (id) => ipcRenderer.invoke('deletarConsulta', id)
}

```

```
});
```

5. Atualizar globals.d.ts

Adicionar métodos na interface API Exemplo: src/types/globals.d.ts

```
interface API {
```

```
  ...
```

```
  criarConsulta(consulta: Consulta): Promise<Consulta>;
```

```
  listarConsultasPorPaciente(id: number): Promise<Consulta[]>;
```

```
  listarTodasConsultas(): Promise<Consulta[]>;
```

```
  atualizarConsulta(consulta: Consulta): Promise<void>;
```

```
  deletarConsulta(id: number): Promise<void>;
```

```
}
```

6. Criar Service no Frontend

src/renderer/services/

Funções assíncronas chamando window.api Exemplo:

src/renderer/services/consultaService.ts

```
import { Consulta } from '../types/Consulta';
```

```
export async function criarConsulta(consulta: Consulta): Promise<Consulta> {
```

```
  return await window.api.criarConsulta(consulta);
```

```
}
```

```
export async function listarConsultasPorPaciente(id: number): Promise<Consulta[]> {
```

```
  return await window.api.listarConsultasPorPaciente(id);
```

```
}
```

```
export async function listarTodasConsultas(): Promise<Consulta[]> {
```

```
  return await window.api.listarTodasConsultas();
```

```
}
```

```
export async function atualizarConsulta(consulta: Consulta): Promise<void> {
```

```
  return await window.api.atualizarConsulta(consulta);
```

```
}
```

```
export async function deletarConsulta(id: number): Promise<void> {
```

```
  return await window.api.deletarConsulta(id);
```

```
}
```

✓ Considerações finais

Tipagem forte via TypeScript e o uso do IPC são seguros, isolados e organizados.

As tabelas SQLite são criados em database.ts, geralmente com CREATE TABLE IF NOT EXISTS

Todas as entidades seguem uma lógica RESTful local: create, list, update, delete.

As páginas React utilizam esses serviços para interagir com o banco por meio de hooks e contextos.

Padrão RESTful: create, list, update, delete

Frontend consome tudo via services